

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Tecnicatura Universitaria en Inteligencia Artificial



PROCESAMIENTO DE IMÁGENES I

IA 4.4

Año: 2025

Integrantes:

- Tapia, Fabrizio Joaquín T-3095/3
- Mezzano, Florencia M-7463/2

Fecha de entrega: 01/12/2025

Docentes:

- Gonzalo Sad
- Juan Manuel Calle
- Joaquín Allione

Informe Técnico

1. Introducción

En este trabajo se desarrolló un pipeline de procesamiento digital de imágenes orientado a la detección, segmentación y clasificación automática de objetos presentes en una escena, utilizando la librería OpenCV y herramientas clásicas de PDI.

A lo largo de dicho trabajo se fueron incorporando distintas etapas de preprocesamiento, filtrado, segmentación, análisis morfológico y extracción de descriptores, resolviendo problemas prácticos que fueron surgiendo en el proceso (ruido, iluminación no uniforme, objetos pegados, detecciones espurias, etc.).

El objetivo final fue obtener una detección robusta de cada objeto, extraer características relevantes que permitan contar, medir y distinguir objetos de forma automática. documentando los distintos intentos, errores y ajustes realizados hasta llegar a la solución final.

2. Objetivos

El objetivo general fue diseñar, implementar y analizar un conjunto de pipelines de procesamiento digital de imágenes que permitan detectar, segmentar y caracterizar automáticamente objetos presentes en distintas escenas, utilizando herramientas básicas de PDI.

Para resolver las diferentes consignas propuestas fue necesario lograr pequeños objetivos más específicos, algunos de ellos fueron:

- Aplicar técnicas de preprocesamiento (conversión a escala de grises, filtros espaciales) para mejorar la calidad de las imágenes de entrada.
- Desarrollar métodos de segmentación basados en umbralización (global, Otsu, etc.) y operaciones morfológicas que separen los objetos de interés del fondo.
- Utilizar componentes conectadas y/o contornos para identificar cada objeto de forma individual.
- Extraer descriptores métricos (área, perímetro, radio equivalente, circularidad, etc.) y, cuando corresponda, información interna de los objetos (por ejemplo, detalles o marcas en su interior).

- Definir criterios de clasificación en función de los descriptores obtenidos, diferenciando tipos de objetos según tamaño, forma o estructura interna.
- Analizar los problemas prácticos encontrados (ruido, objetos pegados, variaciones de iluminación, errores de segmentación) y justificar los ajustes o cambios introducidos en cada etapa.
- Comparar los resultados obtenidos en las distintas consignas, destacando las similitudes y diferencias en las estrategias de procesamiento utilizadas.

Desarrollo

1 Detección y clasificación de monedas y dados

Esta primer etapa consta de tres puntos; el objetivo del Punto A fue separar de manera automática las monedas y los dados del fondo de la imagen *monedas.jpg*, para utilizarlos posteriormente en la clasificación (Punto B) y en el conteo (Punto C).

La imagen presenta varios desafíos: un fondo gris con leves gradientes, objetos metálicos con relieves internos y variaciones de brillo que dificultan la utilización de umbralizaciones globales o adaptativas.

Por estos motivos trabajamos con una estrategia basada en detección de bordes (Canny) combinada con operaciones morfológicas y relleno a partir de contornos externos, que resultó ser la más robusta y estable para este escenario.

1.1.1 Preprocesamiento y Selección de Estrategia (Punto A)

Primero se convierte la imagen a escala de grises y se aplica un suavizado Gaussiano (Blur 11×11). El blur reduce ruido y pequeñas variaciones de iluminación, logrando que el detector de bordes trabaje con una señal más estable.



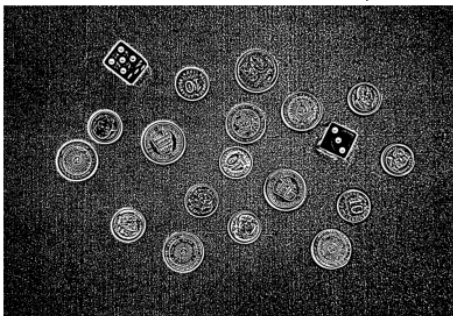
Gris + Blur Gaussiano



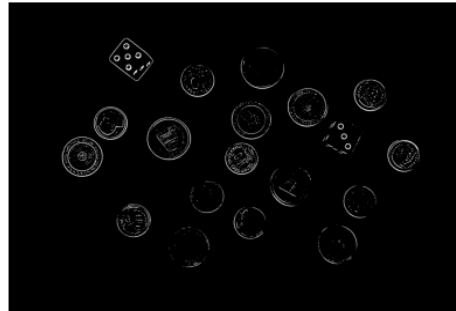
Se probaron diferentes técnicas de umbralización (adaptativa, Otsu y Canny), sin embargo no obtuvimos resultados satisfactorios excepto de Canny, que fue la estrategia seleccionada:

La **umbralización adaptativa** (adaptiveThreshold) produjo máscaras con demasiado ruido, ya que la textura del fondo era interpretada como bordes.

Máscara binaria (umbralización adaptativa)



Máscara binaria (umbralización adaptativa)

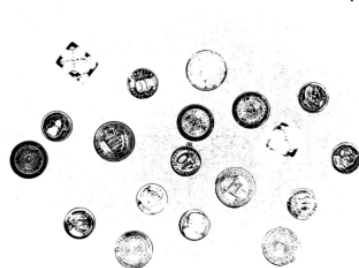


Otsu, incluso combinado con operaciones top-hat, generaba objetos fragmentados debido al fuerte relieve interno de algunas monedas.

Opción B - Imagen tras top-hat



Opción B - Máscara binaria (Otsu sobre top-hat)



Dado que estos métodos no lograban convertir las monedas en regiones sólidas, se descartó la umbralización como estrategia principal.

Canny, luego de descartar la umbralización, se adoptó un enfoque basado en bordes. Se utiliza el detector con umbrales (20, 70)

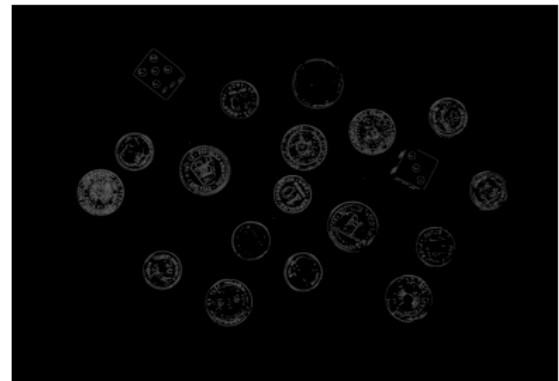
Después de varias pruebas (80–160, 40–120), este rango resultó el más adecuado, ya que con umbrales más altos dejaban los bordes de las monedas incompletos o cortados y umbrales más bajos capturaban contornos más continuos sin introducir ruido excesivo.

Esto permite obtener un mapa de bordes donde los contornos externos de monedas y dados aparecen completos.

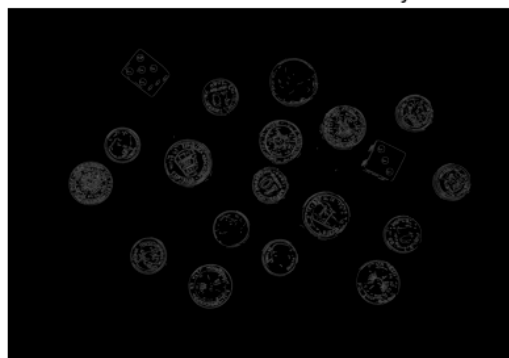
Detección de Bordes con Canny con 80-160



Detección de Bordes con Canny 40-120



Detección de Bordes con Canny 20 - 70

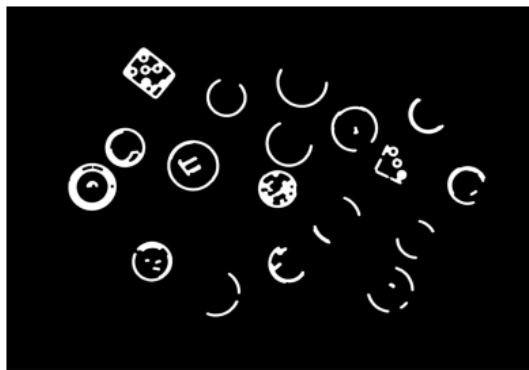


1.1.2 Engrosado de bordes

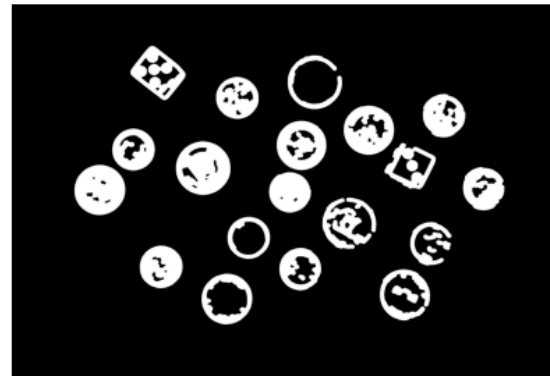
Kernels muy grandes (15×15 , 17×17) cerraban bien, pero empezaban a unir monedas próximas.

Kernels pequeños (3×3) no cerraban huecos relevantes.

Kernel 5×5 con 2 iteraciones ofreció el mejor balance: cerró pequeñas discontinuidades sin fusionar monedas.



Bordes engrosados



Bordes mínimamente engrosados



1.1.3 Contornos externos y filtrado por área

Para evitar que los relieves internos de las monedas aparezcan como huecos, se detectaron únicamente **contornos externos** utilizando RETR_EXTERNAL.

Luego se rellenaron completamente sobre una máscara binaria, generando manchas sólidas que representan cada moneda o dado.

1.1.4 Cierre Final y segmentación de objetos

La máscara generada a partir de contornos se refinó mediante un cierre morfológico final con los parámetros elegidos previamente (kernel 5×5 , 2 iteraciones).

Durante las pruebas observamos que:

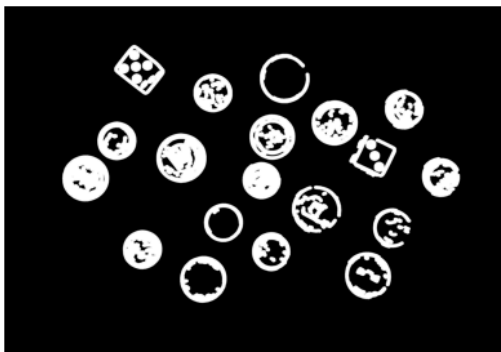
Efecto de los parámetros:

- Kernels más pequeños → los blobs eran incompletos; varias monedas quedaban con huecos.
- Kernels más grandes o más iteraciones → las manchas se volvían demasiado sólidas y podían pegarse entre sí, especialmente en monedas que estaban próximas.

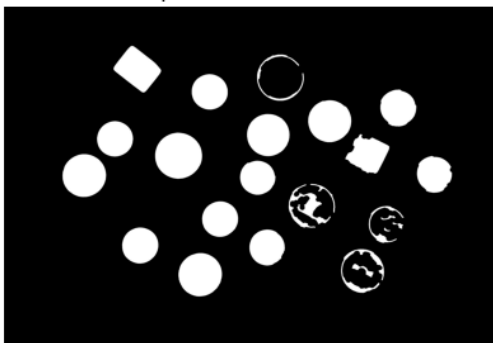
Esto nos permitió elegir el punto justo donde las monedas quedaban sólidas y separadas, pero sin alterarse ni unirse entre sí.

Finalmente, la máscara refinada se aplicó sobre la imagen original mediante una operación bitwise, obteniendo una segmentación clara donde solo quedan visibles las monedas y los dados.

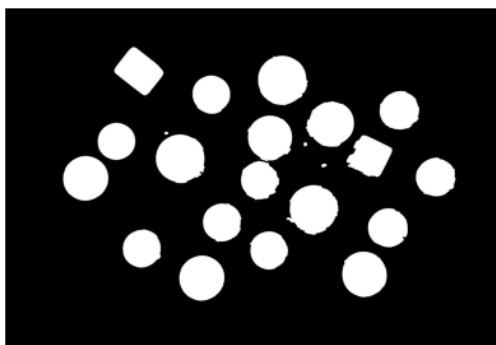
Máscara rellenada



Máscara a partir de contornos rellenos



Máscara con contornos filtrados



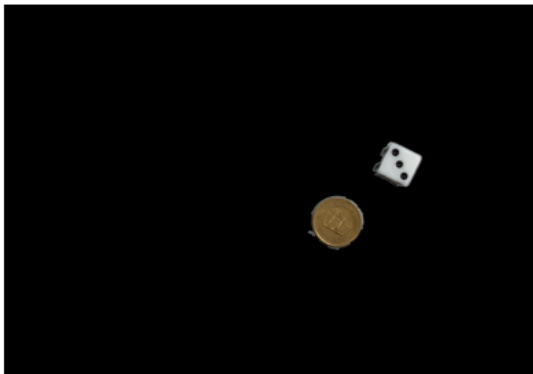
1.2 Clasificación de monedas (Punto B)

Una vez obtenidos los contornos filtrados de todos los objetos (monedas y dados) a partir de la segmentación del Punto A, se avanzó hacia la clasificación automática de las monedas según su tamaño y la separación explícita entre monedas y dados.

Al inicio intentamos separar los diferentes objetos calculando el área o por circularidad, pero no obtuvimos los resultados esperados; al no estar finamente segmentados los objetos tenían el área muy grande o no eran perfectamente circulares, por lo que monedas eran identificadas como dados.

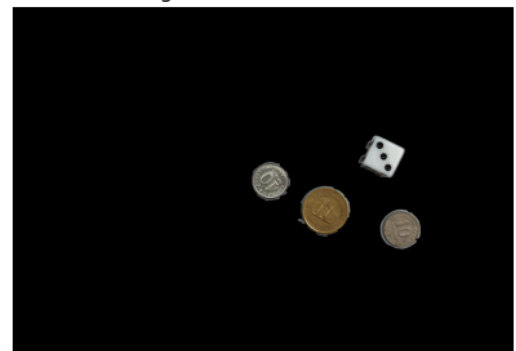
Esto lo fuimos descubriendo al mostrar por pantalla el área de cada objeto.

Segmentación de dados



```
(venv) PS C:\Users\Usuar
ario/OneDrive/Documentos
0.7900893425624737
(venv) PS C:\Users\Usuar
ario/OneDrive/Documentos
Monedas detectadas: 15
Dados detectados: 4
0.8911146617257227
0.842965734170444
0.8514854469848142
0.8579803912021264
0.7840066335041412
0.8259475573319357
0.6930317375527543
0.7649475228542153
0.8177869486580018
0.8948619268315701
0.6176399252909781
0.7849827025608284
0.8951549782467443
0.8266898339308801
0.7945980766908828
0.8023093435214634
0.8746777848343101
0.8202199442996099
0.7900893425624737
```

Segmentación de dados



Finalmente decidimos separar monedas y dados usando una métrica de roundness (redondez) y aplicar k-means sobre el radio equivalente de las monedas para agruparlas en tres tipos.

1.2.1 Cálculo de descriptores y métrica de roundness

Para cada contorno de la lista `contours filtered` se calcularon el área del contorno (`area`), el perímetro (longitud del contorno (`perim`)), centroide del objeto (`cx`, `cy`), radio equivalente de un círculo con la misma área (`radio_eq`), círculo mínimo envolvente del contorno mediante `cv2.minEnclosingCircle`, con radio `r` (`area_circ`).

A partir de ese círculo se calcula su área. Con estos valores se definió la métrica de `roundness` como `area/area_circ`.

Esta medida indica qué tan bien “llena” el objeto el círculo mínimo que lo envuelve. Valores cercanos a 1 indican objetos muy redondos (monedas); valores menores corresponden a formas más cuadradas o irregulares (dados).

Toda esta información se almacenó en una lista de diccionarios objetos, manteniendo para cada uno: contorno, área, perímetro, centroide, radio equivalente, `roundness` y radio del círculo mínimo.

Con dichos descriptores calculados, se definió un umbral de `roundness` y se separaron las clases; objetos con `roundness` ≥ 0.8 se consideraron monedas, si estaban por debajo, se consideraron dados. Dicho umbral se eligió empíricamente.

De este modo, la separación entre monedas y dados dejó de depender del área o sólo de la circularidad y pasó a depender de la forma calculada por más de un descriptor, lo que resultó más robusto frente a pequeñas deformaciones o variaciones de tamaño.

1.2.2 Clasificación de monedas en 3 tipos mediante k-means

Una vez aisladas las monedas, el siguiente paso fue dividir las en 3 tipos según su tamaño. En lugar de fijar umbrales a mano sobre el área o el radio equivalente, se utilizó `k-means` para que el propio algoritmo agrupe las monedas en tres clusters.

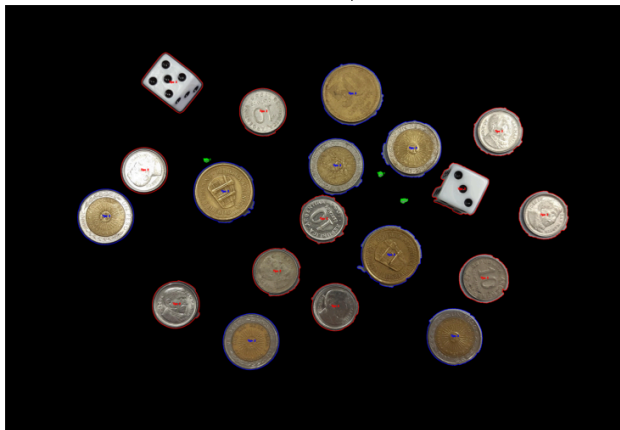
Construimos un vector con el radio equivalente de cada moneda, definimos los parámetros de `k-means` (criterio de parada, número de iteraciones máximas, etc.) y el valor de $K = 3$ se eligió porque, a partir de la inspección visual de la imagen se identificaron tres tamaños distintos de monedas.

Los centroides de los clusters fueron ordenados de menor a mayor para poder etiquetar, el cluster con menor radio equivalente como Moneda 1, el siguiente como Moneda 2 y el mayor como Moneda 3, siendo esta última la más grande.

De esta forma la clasificación no depende de umbrales fijados manualmente sino de los grupos que el propio algoritmo detecta de los datos.

También se filtraba mucha basura del fondo, por lo que debimos ajustar el área mínima de segmentación del Punto A y así poder clasificar correctamente cada objeto detectado.

Monedas clasificadas por tamaño



Monedas por tipo y dados

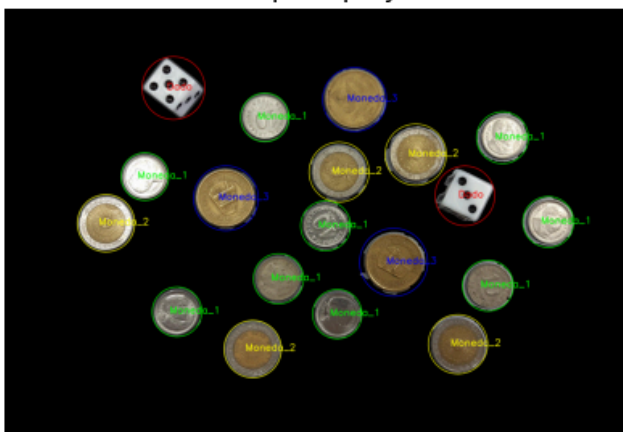


1.2.3 Visualización y conteo de monedas por tipo

Para la visualización final se dibujó, sobre una copia de la imagen segmentada (segm_contours) un círculo de color alrededor de cada moneda, utilizando el radio del círculo mínimo (r), un texto grande con la etiqueta de tipo: “Moneda_1”, “Moneda_2” o “Moneda_3” y los dados se marcaron con un círculo rojo y la palabra “Dado”.

También se realizó un conteo automático, lo que permitió obtener el número de monedas de cada tipo.

Monedas por tipo y dados



```
=== Conteo automático de monedas por tipo ===  
Moneda_1: 9  
Moneda_2: 5  
Moneda_3: 3
```

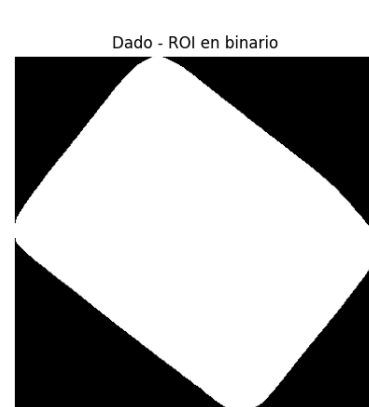
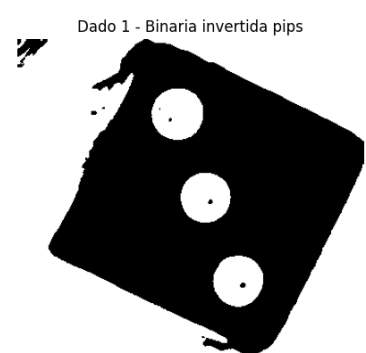
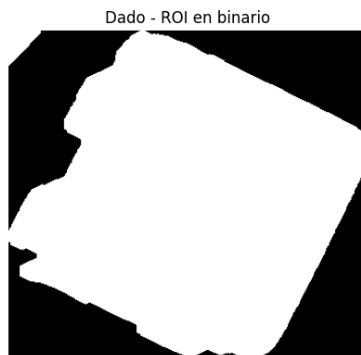
1.2.4 Conteo de dados y valoración de sus caras (Punto C)

En el Punto C el objetivo fue identificar los dados presentes en la escena y determinar, para cada uno, la cantidad de puntos (pips) en su cara visible, es decir, su valor entre 1 y 6.

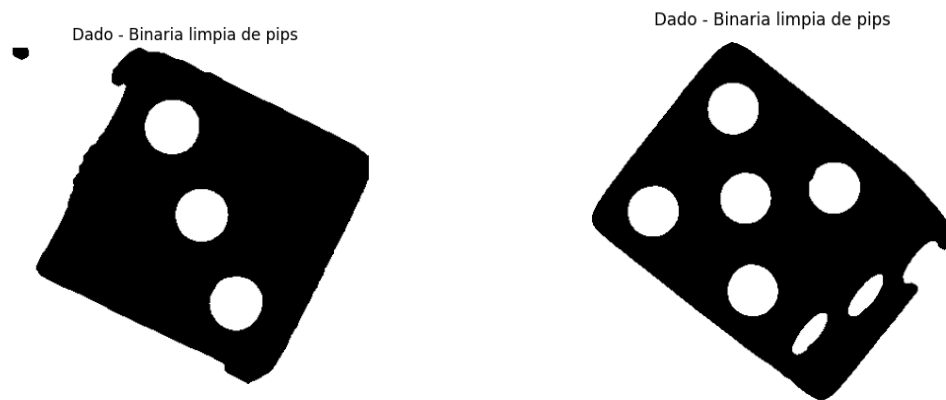
A partir de la misma máscara binaria utilizada en los puntos anteriores, y de la clasificación monedas/dados, se tomaron como dados aquellos contornos, con área dentro de un rango compatible con el tamaño esperado del dado y con circularidad más baja (formas cuadradas), según el filtro aplicado en el Punto B.

1.2.5 Definición de ROIs por dado

Para cada contorno etiquetado como dado se calculó su bounding box y la máscara se utilizó para aislar el dado del fondo. Esto permitió trabajar solo sobre los píxeles del dado. Dentro de cada ROI se aplicó suavizado Gaussiano para reducir ruido y sobre eso umbralización binaria inversa con Otsu, de modo que los puntos quedaran como blobs blancos sobre el fondo negro.



Y luego se realizó una limpieza morfológica en dos pasos, un cierre con kernel elíptico 7x7 para rellenar pequeños huecos en los blobs y una apertura con un kernel más chico de 3x3 para eliminar ruido aislado. Así se obtuvo una imagen binaria con blobs blancos compactos.



1.2.6 Selección de pips válidos y descarte de blobs laterales

Sobre la Región de Interés (ROI) final, se identificaron inicialmente todos los contornos, lo que incluyó no solo los puntos (pips) reales, sino también elementos no deseados como bordes laterales del dado, sombras y residuos de textura. Para aislar únicamente los puntos válidos, se implementó una secuencia de filtros. El primer filtro fue por **Área**, descartando contornos demasiado pequeños o excesivamente grandes. Seguidamente, el filtro por **Redondez** (*Roundness*) calculó el círculo mínimo envolvente para cada contorno, usando un umbral para seleccionar solo aquellos lo suficientemente circulares. Finalmente, el filtro por **Relación de Aspecto** (*Aspect Ratio*), basado en el *bounding box* del contorno, aplicó un umbral para asegurar que el punto fuera casi cuadrado o circular. Estos filtros fueron cruciales para eliminar elementos no deseados, como los *blobs* laterales, que generalmente poseen un área fuera del rango normal de un pip, exhiben menor redondez y son más alargados (alta relación de aspecto).



El valor de cada dado se definió directamente como la cantidad de pips válidos detectados.

```
329
330     valores_datos.append(
331         d["valor"] = num_pips
332
333     # Visualización de la
334     img_cont = cv2.cvtColor(
335         img, cv2.COLOR_BGR2GRAY)
336     mostrar(img_cont, f"Dado 1 - Contornos de pips válidos")
```

PROBLEMS OUTPUT TERMINAL POR

(venv) PS C:\Users\Usuario\OneDrive\Documents\TUIA\4to

Dados detectados: 2

c:\Users\Usuario\OneDrive\Documents\TUIA\4to

error in future. Ensure you extract

cl = int(labels[i])

=== Conteo automático de monedas p

Moneda 1: 9

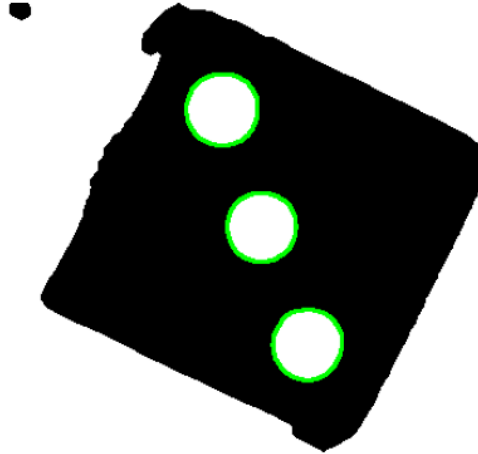
Moneda 2: 5

Moneda 3: 3

Pips detectados en el dado 1: 3

█

Dado 1 - Contornos de pips válidos



```
328     print(f"Pips detectados: {num_pips}")
329
330     valores_datos.append(
331         d["valor"] = num_pips
332
333     # Visualización de la
334     img_cont = cv2.cvtColor(
335         img, cv2.COLOR_BGR2GRAY)
336     mostrar(img_cont, f"Dado 2 - Contornos de pips válidos")
```

PROBLEMS OUTPUT TERMINAL PC

(venv) PS C:\Users\Usuario\OneDrive\Documents\TUIA\4to

Dados detectados: 2

c:\Users\Usuario\OneDrive\Documents\TUIA\4to

error in future. Ensure you extract

cl = int(labels[i])

=== Conteo automático de monedas

Moneda 1: 9

Moneda 2: 5

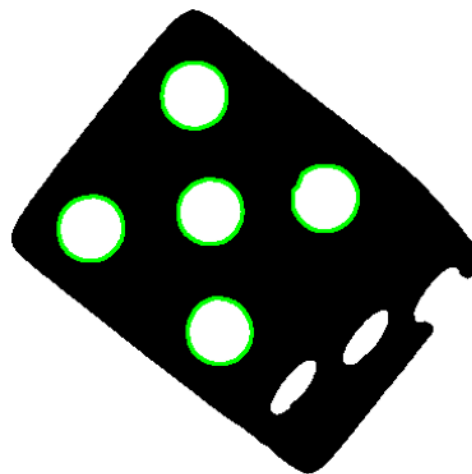
Moneda 3: 3

Pips detectados en el dado 1: 3

Pips detectados en el dado 2: 5

█

Dado 2 - Contornos de pips válidos



2 Detección de patentes

Durante esta segunda etapa del trabajo desarrollamos dos implementaciones distintas para la detección automática. El primero fue un pipeline más complejo, basado en bordes, morfología y selección de regiones por heurísticas geométricas. Ese enfoque terminó siendo poco eficiente, además que no logramos encontrar los parámetros para ajustarlo

correctamente, por lo que finalmente decidimos descartarlo y comenzar nuevamente desde una estrategia más simple, más controlada y mejor adaptada al tipo de imágenes que teníamos.

2.1.1 Pipeline heurístico basado en Sobel y morfología (enfoque descartado)

El primer enfoque buscaba resolver todo de una sola pasada: obtener bordes con Sobel, cerrar regiones con morfología, detectar contornos, filtrar por forma y luego refinar la región seleccionada con un submódulo interno.

Fue una idea que fuimos construyendo de a poco, ajustando parámetros y umbrales, con un preprocesamiento basado en blur bilateral, Sobel y umbral de Otsu, seguido de morfología agresiva con kernels horizontales largos para consolidar regiones rectangulares.

Sobre esa salida se analizaron los contornos externos, aplicando filtros geométricos (relación ancho/alto, área, posición en la imagen), junto con un filtro de densidad interna de bordes y, en los casos en que el recorte resultaba demasiado grande, un refinamiento secundario del ROI.

Finalmente, se intentaba usar la segmentación de caracteres como validación final de que la región fuera efectivamente una patente. Sin embargo, los errores fueron consistentes: se detectaban demasiados contornos irrelevantes (reflejos, paragolpes, zonas iluminadas e incluso sombras aparecían como “posibles patentes”), el método fallaba especialmente cuando el auto estaba lejos porque los umbrales pensados para imágenes grandes dejaban de funcionar para patentes pequeñas, y la segmentación de caracteres no corregía nada cuando el ROI ya estaba mal detectado, llegando a dibujar “caracteres” sobre ruido.

En síntesis, el método resultó excesivamente dependiente de heurísticas y poco robusto: funcionaba bien en algunos casos aislados, pero fallaba de manera sistemática en otros, motivo por el cual decidimos descartarlo y volver a empezar desde cero con un pipeline mucho más sencillo, interpretable y controlable.





2.1.2 Pipeline simplificado basado en textura vertical (enfoque elegido)

Para la nueva versión eliminamos toda la complejidad innecesaria del primer intento y retomamos el problema desde cero.

La idea central fue apoyarse en una propiedad muy clara de la patente argentina: presenta barras verticales bien marcadas, correspondientes a caracteres claros, sobre un fondo relativamente oscuro y plano.

A partir de esto, el nuevo método se construyó alrededor de unos pocos pasos clave: binarizar correctamente la imagen, detectar rectángulos razonables, evaluar la textura vertical en su interior, elegir el mejor candidato y, finalmente, recortar la patente para validarla visualmente.



2.1.3 Preprocesamiento simple y obtención de contornos

El preprocesamiento se redujo al mínimo necesario. Primero se aplicó una reducción homogénea de tamaño para trabajar siempre en una escala controlada; luego se convirtió la imagen a escala de grises y se utilizó umbralado de Otsu para separar zonas claras de zonas oscuras, obteniendo una binarización limpia sin depender de parámetros finos. A diferencia del enfoque anterior, los filtros morfológicos agresivos se eliminaron por completo: la morfología se limitó a operaciones mínimas, evitando el uso de kernels gigantes que en la práctica generaban más artefactos que beneficios. Sobre la imagen binarizada se extrajeron únicamente contornos externos y se aceptó cualquier región aproximadamente rectangular.

El filtrado por forma se dejó deliberadamente relajado: solo se descartaron regiones extremadamente alargadas o extremadamente altas, y regiones demasiado pequeñas como para contener una patente. Con esto se buscó evitar el error del pipeline anterior, donde la patente podía ser descartada prematuramente por criterios geométricos demasiado rígidos.

2.1.4 Evaluación de textura interna y selección del candidato

El cambio más importante del nuevo enfoque fue la forma de decidir qué rectángulo es realmente una patente.

En lugar de seguir acumulando heurísticas geométricas, se explotó directamente la textura interna: una patente verdadera presenta muchas transiciones verticales debidas a los bordes de las letras y números.

Sobre cada región candidata se calculó el gradiente vertical utilizando Sobel-X y se tomó la media del valor absoluto del gradiente como medida de “energía vertical”. Las regiones que no presentan barras verticales pronunciadas —por ejemplo, paragolpes, reflejos suaves, sombras, carteles sin texto marcado o chapas lisas— quedan automáticamente penalizadas porque su energía vertical es baja.

El criterio de decisión se volvió entonces muy simple: entre todos los candidatos válidos, se elige como patente aquella región que presenta el mayor valor de energía vertical, es decir, la que más “letras” aparenta contener.

Este criterio resultó mucho más estable y directo que la combinación de filtros geométricos utilizada en el primer enfoque.

2.1.5 Errores detectados y ajustes del nuevo método

Durante el desarrollo del nuevo método aparecieron algunos errores recurrentes que llevaron a ajustes puntuales.

En varias imágenes, la parte superior del auto (capot, parabrisas, reflejos del sol) generaba candidatos falsos con cierta textura vertical. Para mitigarlo, se priorizó la búsqueda y selección de candidatos en la mitad inferior de la imagen, donde se espera que se encuentre la patente; este simple sesgo espacial eliminó la mayoría de esos falsos positivos sin añadir complejidad significativa.

En otros casos, ciertas zonas brillantes o reflejos verticales producían “falsas letras” al aplicar Sobel. La solución fue calcular la energía vertical sobre el recorte original, y no sobre versiones excesivamente procesadas morfológicamente, lo que redujo drásticamente la cantidad de candidatos espurios.

Finalmente, cuando la patente aparecía muy pequeña, el valor de energía vertical se reducía y algunos filtros geométricos iniciales podían descartarla inadvertidamente. Para evitarlo, se relajaron aún más estos filtros de forma, de manera que el método casi no descarta contornos por criterios geométricos salvo en casos extremos.

El resultado final fue un pipeline mucho más sencillo, interpretable y robusto, centrado en la propiedad clave de la escena: la fuerte textura vertical de los caracteres de la patente.

2.1.6 Separación y lectura de caracteres

Una vez detectado el rectángulo final de la patente, se trabajó específicamente en la segmentación interna de los caracteres. Esta fue una etapa completamente nueva dentro del segundo enfoque y requirió diseñar un pipeline propio para pasar de la región de la patente a componentes individuales que puedan interpretarse como letras o números.

El objetivo fue obtener una binarización lo bastante limpia como para separar cada carácter en una componente conectada distinta, evitando tanto la pérdida de información como la fusión indeseada de letras entre sí.

2.1.7 Conversión a gris y binarización limpia

En esta etapa se probaron distintas variantes de filtrado previo (Gaussian, median y bilateral) y se concluyó que la combinación de un suavizado gaussiano mínimo seguido de umbralado de Otsu ofrecía el mejor compromiso entre eliminación de ruido y preservación de bordes.

Este paso resultó crítico porque cualquier píxel negro de más podía conectar letras contiguas y transformar varias letras en un único bloque binario.

Por el contrario, una binarización demasiado conservadora corría el riesgo de “comerse” trazos finos de algunos caracteres.

El ajuste de este preprocesamiento fue, por lo tanto, central para que la segmentación por componentes conectadas funcionara correctamente.

2.1.8 Morfología mínima

La morfología aplicada sobre la patente se redujo al mínimo posible. Se utilizaron únicamente kernels muy pequeños, del orden de 1×1 o 1×2 , con el objetivo de corregir detalles extremadamente locales sin alterar la forma general de los caracteres.

Durante las pruebas se observó que cualquier operación más fuerte —incluso una apertura con un kernel 2×2 — comenzaba a deformar las letras, adelgazando trazos o rellenando espacios internos, lo que empeoraba la segmentación posterior.

Por este motivo, la morfología dejó de ser una herramienta principal y pasó a ser un ajuste fino muy controlado.

2.1.9 Segmentación por componentes conectados y filtros adaptativos

La segmentación final se realizó exclusivamente mediante componentes conectados, evitando heurísticas más agresivas y manteniendo el esquema lo más simple posible.

A cada componente detectada se le midieron sus características geométricas básicas, como altura, ancho, área, relación de aspecto y posición relativa dentro de la patente, y luego se ordenaron todas las componentes de izquierda a derecha para respetar el orden natural de lectura.

A diferencia del primer enfoque, los filtros en esta etapa fueron deliberadamente muy relajados: en vez de fijar umbrales rígidos, se tomaron como referencia las medianas

globales de altura y ancho para descartar solo aquellos elementos claramente fuera de escala. Esto permitió conservar incluso caracteres algo deformados o parcialmente degradados, sin perder letras válidas por criterios excesivamente estrictos.

2.1.10 Errores durante la segmentación de caracteres

Durante esta etapa apareció un problema estructural que fue necesario abordar explícitamente: muchas letras aparecían conectadas entre sí por pequeños puentes de píxeles. A simple vista la patente parecía perfectamente legible, pero en la imagen binaria esos caracteres formaban una única componente grande, lo que impedía contar y separar correctamente cada símbolo. Este efecto se observó con mayor frecuencia en los bordes de la chapa y en las uniones entre caracteres adyacentes, donde el contraste local y las sutilezas de la binarización favorecían la aparición de “pegamentos” negros que unían letras que deberían estar separadas. La combinación de un blur gaussiano suave, un umbralado cuidadoso y una morfología mínima fue clave para reducir la cantidad de conexiones artificiales, aunque se mantuvo como una limitación conocida del método: cuando los caracteres están demasiado próximos o el escaneo es de baja calidad, la segmentación perfecta letra a letra puede verse comprometida.



3. Conclusiones

El trabajo permitió integrar de manera práctica gran parte de las herramientas vistas en la materia, pasando desde el preprocesamiento y la segmentación hasta el análisis morfológico y el diseño de descriptores para clasificación. En la primera parte, se logró una segmentación robusta de monedas y dados combinando detección de bordes con Canny, operaciones morfológicas y filtrado por contornos, lo que permitió obtener máscaras limpias incluso ante relieves internos e iluminación no uniforme. Sobre esa base, el uso de descriptores geométricos (área, perímetro, radio equivalente, roundness) y k-means hizo posible separar automáticamente monedas y dados, distinguir tres tipos de monedas por tamaño y contar los puntos de cada dado a partir de un procesamiento específico dentro de la ROI.

En la segunda parte, el desarrollo de dos enfoques para detección de patentes mostró claramente el valor de la simplicidad bien orientada frente a pipelines heurísticos: el primer método, basado en mucha morfología y filtros geométricos, resultó difícil de ajustar; el segundo, centrado en la textura vertical de los caracteres, en una binarización cuidada y en criterios suaves de forma, produjo un pipeline más corto e interpretable. Sobre la patente recortada, la segmentación de caracteres mediante componentes conectadas y morfología mínima permitió aislar letras y números de manera razonable, aunque se identificaron limitaciones cuando los caracteres aparecen muy cercanos o conectados entre sí.

En resumen, el trabajo evidencia que, para muchos problemas de procesamiento de imágenes clásicos, no es necesario recurrir a soluciones complejas o de “caja negra”: combinar bien unos pocos bloques (binarización adecuada, buen diseño de descriptores, morfología medida y decisiones apoyadas en la estructura real de la escena) suele dar soluciones más estables y fáciles de depurar. Como líneas futuras, sería natural extender estos pipelines a conjuntos de imágenes más variados, incorporar etapas de validación y explorar enfoques híbridos donde estas etapas clásicas se integren con modelos de aprendizaje automático u OCR para mejorar la lectura automática de patentes y la clasificación de objetos.